

Strings in Python

part two

7. String methods

String methods

dot notation

string methods are applied using the dot notation

The string methods are applied on the string instance, using the *dot notation*. As follows:

```
1 original_hello = "Hello World!"
2 upper_hello = original_hello.upper()
3 print(upper_hello)
```

HELLO WORLD!

Contrast this with how the built-in functions are called:

```
1 original_hello = "Hello World!"
2 str_length = len(original_hello)
3 print(str_length)
```

String methods

dot notation

The string methods are applied using the *dot notation*.

```
1 original_hello = "Hello World!"
2 upper_hello = original_hello.upper()
3 print(upper_hello)
```

HELLO WORLD!

Contrast this with how a function is called:

```
1 def get_first_last_char(sentence):
2     return sentence[0] + sentence[-1]
3
4 original_hello = "Hello World!"
5 first_last = get_first_last_char(original_hello)
6 print(first_last)
```

H!

String methods

When introducing some of the string methods, we will use the following division of the methods:

- ▶ Methods without arguments
- ▶ Methods with arguments
- ▶ Methods that return a boolean

Python documentation

<https://docs.python.org/3/library/stdtypes.html#text-sequence-type-str>

String Methods

Strings implement all of the [common](#) sequence operations, along with the additional methods described below.

Strings also support two styles of string formatting, one providing a large degree of flexibility and customization (see [str.format\(\)](#), [Format string syntax](#) and [Custom string formatting](#)) and the other based on C `printf` style formatting that handles a narrower range of types and is slightly harder to use correctly, but is often faster for the cases it can handle ([printf-style String Formatting](#)).

The [Text Processing Services](#) section of the standard library covers a number of other modules that provide various text related utilities (including regular expression support in the [re](#) module).

str.capitalize()

Return a copy of the string with its first character capitalized and the rest lowercased.

Changed in version 3.8: The first character is now put into titlecase rather than uppercase. This means that characters like digraphs will only have their first letter capitalized, instead of the full character.

8. String methods without arguments

String methods without arguments

`upper()/lower()`

`upper()` and `lower()` returns a new string with the same content as the string instance which the method is applied to, except that it has been upper-cased/lower-cased.

```
1 original_hello = "Hello World!"
2 upper_hello = original_hello.upper()
3 print("Original string:", original_hello)
4 print("Upper-cased string:", upper_hello)
5 print("Lowered string:", original_hello.lower())
```

Original string: Hello World!

Upper-cased string: HELLO WORLD!

Lowered string: hello world!

String methods without arguments

upper()/lower()

```
1 original_hello = "Hello World!"  
2 upper_hello = original_hello.upper()
```

Note how the dot notation is used, and that the method does not take any arguments and thereby it is written with an empty parenthesis.

(The statement about no arguments might be argued about, if you later dig very deep into Python, but for most applications, it's a very reasonable simplification.)

String methods without arguments

strip()

`strip()` returns a new string where white space has been removed (and equivalents such as newline) from the beginning and end of a string.

```
1 hello_with_space = " Hello World!\n"
2 print "*" + hello_with_space + "*"
3 print "-----"
4 print "*" + hello_with_space.strip() + "*"

```

* Hello World!

*

Hello World!

9. String methods with arguments

String methods with arguments

Replace takes two arguments/parameters: The string to replace, and the string with which to replace it. (As usual, the original string stays the same, and a new, changed string is returned.)

```
1 original_hello = "Hello World!"
2 replaced_hello = original_hello.replace("!", "?")
3 print(replaced_hello)
4 print(original_hello)
```

Hello World?

Hello World!

You can also replace a longer string sequence. ("" means empty string, so to replace something with an empty string, has the effect that it will be removed.)

```
1 original_hello = "Hello World!"
2 print(original_hello.replace("Hello ", ""))
```

World!

String methods with arguments

count and find

`count` takes one argument: The string for which to search the *number of occurrences* in the string instance on which it is applied.

```
1 print ("Hello world!".count("o"))
2 print ("Hello world!".count("m"))
3 print ("Hello world!".count("wo"))
```

2

0

1

`find` also takes one argument: The string for which to return the *index* where it is found in the string instance on which it is applied.

```
1 print ("Hello world!".find("o"))
2 print ("Hello world!".find("m"))
3 print ("Hello world!".find("wo"))
```

4

-1

6

String methods with arguments

count and find

Both methods takes one argument, and returns an int. In the first case, this int contains the number of occurrences.

In the second case, the int contains an index in the string. When -1 is returned, it means that the string was not found in the string that was searched.

```
1 print("Hello world!".count("o"))
```

2

```
1 print("Hello world!".find("o"))  
2 print("Hello world!".find("m"))
```

4

-1

9. String methods that return a boolean

String methods that return a boolean

String methods that return a boolean can be used for testing properties of the strings.

`endswith()` takes one argument, and tests if the string ends with that argument. (There is also a `startswith()` method.)

`isdigit()` returns true if all characters in the string are digits (and there is at least one character in the string), false otherwise. It does not take any arguments.

```
1 print("Hello world!".endswith("world!"))
2 print("Hello world!".isdigit())
3 print("1000".isdigit())
```

True

False

True

String methods that start with 'is'

and that return a boolean

`str.isalnum()`

Return true if all characters in the string are alphanumeric and there is at least one character, false otherwise. `c.isnumeric()`.

`str.isalpha()`

Return true if all characters in the string are alphabetic and there is at least one character, false otherwise. `Alp` category property being one of "Lm", "Lt", "Lu", "Ll", or "Lo". Note that this is different from the "Alphabetic" pr

`str.isdecimal()`

Return true if all characters in the string are decimal characters and there is at least one character, false other ZERO. Formally a decimal character is a character in the Unicode General Category "Nd".

`str.isdigit()`

Return true if all characters in the string are digits and there is at least one character, false otherwise. Digits covers digits which cannot be used to form numbers in base 10, like the Kharosthi numbers. Formally, a digit i

`str.isidentifier()`

Return true if the string is a valid identifier according to the language definition, section [Identifiers and keywoi](#)

Use `keyword.iskeyword()` to test for reserved identifiers such as `def` and `class`.

`str.islower()`

Return true if all cased characters [\[4\]](#) in the string are lowercase and there is at least one cased character, false

`str.isnumeric()`

Return true if all characters in the string are numeric characters, and there is at least one character, false othe e.g. U+2155, VULGAR FRACTION ONE FIFTH. Formally, numeric characters are those with the property value N

`str.isprintable()`

Return true if all characters in the string are printable or the string is empty, false otherwise. Nonprintable cha space (0x20) which is considered printable. (Note that printable characters in this context are those which s `sys.stdout` or `sys.stderr`.)

`str.isspace()`

Return true if there are only whitespace characters in the string and there is at least one character, false other